localization patch validation general

view
/testbed
L1–200

view
/testbed/.

1 ls 2 sed 3 sed 4 sed 5 str_replace ✗

.../autoreload.py
str_1, str_2

6

7

view
.../autoreload.py
L1–200

8 view
.../autoreload.py
L200–400

9 str_replace ✗
.../autoreload.py
str_1, str_3

10 str_replace ✗ .../autoreload.py
str_1, str_3

11

python
.../n", re.m)new='''def get_child_arguments(): """ return the executable. this contains a workaround for windows if the executable is reported to not have the.exe extension which can cause bugs on reloading. detect when python was started with -m. the documented way to determine this is to inspect the top-level __main__ module's __spec__. if __main__.__spec__ is not none and its parent is a non-empty string, then python was started with -m <parent>. fall back to the previous heuristic used for older python environments where __spec__ may not be set. """ import django.__main__ django_main_path = path(django.__main__.__file__) py_script = path(sys.argv[0]) args = [sys.executable] + ['-w%s' % o for o in sys.warnoptions] # preferred: use __main__.__spec__ to detect -m invocation. main_mod = sys.modules.get('__main__') if main_mod is not none: main_spec = getattr(main_mod, '__spec__', none) if main_spec is not none: parent = getattr(main_spec, 'parent', none) # if parent is a non-empty string, python was invoked as `python -m <parent>`. if parent: args += ['-m', parent] args += sys.argv[1:] return args # fallback for environments where __spec__ is not set on __main__. if py_script == django_main_path: # the server was started with `python -m django runserver`. args += ['-m', 'django'] args += sys.argv[1:] elif not py_script.exists(): # sys.argv[0] may not exist for several reasons on windows. # it may exist with a.exe extension or have a -script.py suffix. exe_entrypoint = py_script.with_suffix('.exe') if exe_entrypoint.exists(): # should be executed directly, ignoring sys.executable. # todo: remove str() when dropping support for py37. # args parameter accepts path-like on windows from python 3.8. return [str(exe_entrypoint), *sys.argv[1:]] script_entrypoint = py_script.with_name('%s-script.py' % py_script.name) if script_entrypoint.exists(): # should be executed as usual. # todo: remove str() when dropping support for py37. # args parameter accepts path-like on windows from python 3.8. return [*args, str(script_entrypoint), *sys.argv[1:]] raise runtimeerror('script %s does not exist.' % py_script) else: args += sys.argv return args'''if not pattern.search(s): print('pattern not found, aborting')else: s2=pattern.sub(new,s, count=1) p.write_text(s2) print('replaced')

14

sed 12

sed 15

13

pytest 16 cat 17 python

/testbed/repro.py

18 grep 19 true 19

sed

20

think

23

26

sed 21

submit

24

27

grep 22

22

rm

25

git

25

git

25

25